KHULNA UNIVERSITY OF ENGINEERING & TECHNOLOGY

Department of Computer Science and Engineering

Project Report

On

Book Exchange Management System

Course Title: Software Engineering and Project Management Laboratory

Course No: CSE 3220

Submitted To,

Mr. Safin Ahmmed
Lecturer,
Department of Computer Science and Engineering,
Khulna University of Engineering and Technology

Mr. Nabil Faiyaz Sadi
Lecturer,
Department of Computer Science and Engineering,
Khulna University of Engineering and Technology

Submitted By,

Sadia Mostafa (2107095)
Prothom Ghosh (2107097)

Lab Group : B2
Year : 3rd Term: 2nd
Date : 05/04/2026

Book Exchange SEPM

Project Report

Focused on engineering lifecycle, architecture, security, testing, CI, deployment, and teamwork

Date: April 5, 2026

Stack: Java 17, Spring Boot 4, PostgreSQL, Thymeleaf, Spring Security, WebSocket, Docker, GitHub Actions, Render

Book Exchange SEPM is a full-stack Spring Boot project built not only to deliver features, but to practice complete software engineering workflow. The project focuses on layered architecture, REST API design, role-based security, maintainability, testing, GitHub collaboration, CI support, and deployment readiness.

The book exchange domain was chosen because it naturally involves authentication, authorization, database relationships, workflow control, and real-time communication. These features helped the team apply stronger engineering ideas such as modular design, design patterns, secure request handling, and clear project structure for a two-person team.

This report explains the project from that engineering lifecycle perspective. It highlights the objective, architecture, security model, full-stack integration, testing approach, CI workflow, and deployment support to show that the project is more than a functional application.

1. Objective

The main objective of Book Exchange SEPM was to practice the complete lifecycle of an industry-style software project. The visible features were important, but the bigger goal was to build a system that shows how real engineering work should be organized from design to deployment.

More specifically, the project aimed to show:

1. a maintainable backend structure
2. secure authentication and role-based access control
3. clearly organized REST APIs
4. separation of business logic from controllers
5. use of software design patterns where they add clarity
6. integration of server-rendered frontend pages with backend logic
7. real-time communication through WebSocket
8. testing across unit, integration, and security levels
9. GitHub-based workflow with automated checks
10. deployment readiness through Docker and Render configuration

This objective matters because the project should be judged not only by features, but by whether another developer could understand, test, extend, review, and deploy it.

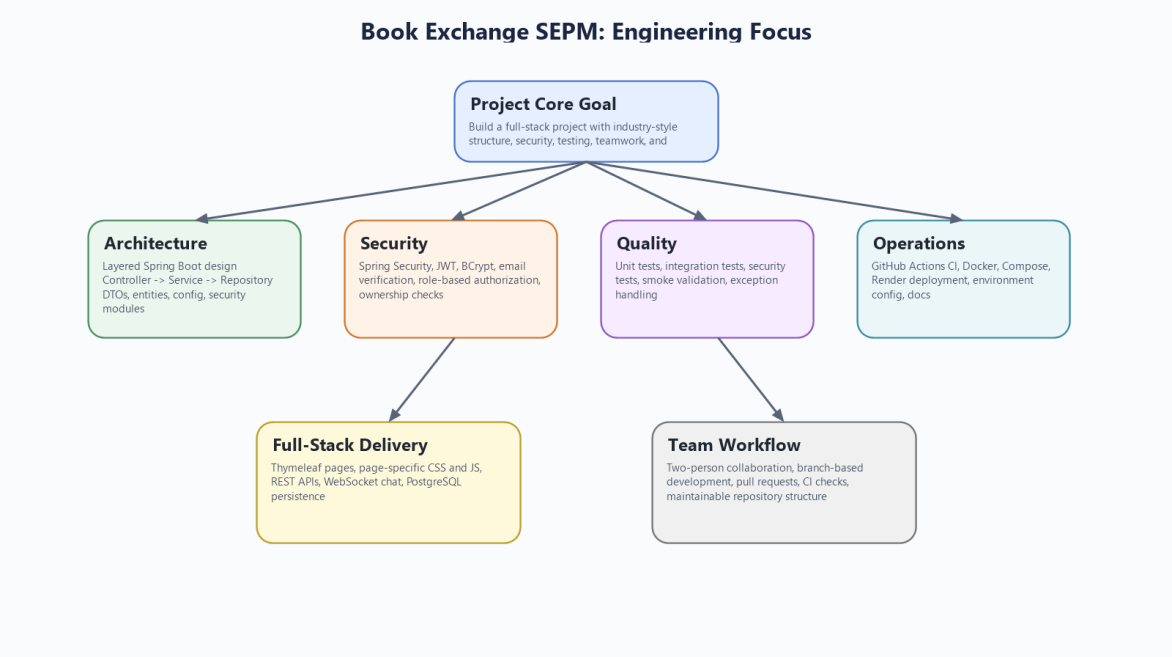

Figure 1. The project was designed around engineering quality, not only end-user features.

2. Introduction

In many student projects, the main focus stays on visible features, while the engineering process behind those features receives much less attention. In real software development, that balance is different. A system is judged not only by what it can do, but also by how safely and sustainably it was built. Because of that, Book Exchange SEPM was designed to feel like a real software product in terms of lifecycle thinking, not only in terms of output.

The selected domain, a book exchange platform, supports that goal well. It is simple enough to explain clearly, but rich enough to require meaningful engineering decisions. Once users can exchange books, the system must handle accounts, roles, ownership, requests, approvals, notifications, real-time chat, and completion rules.

The team used this project to explore how a full-stack Spring Boot application should be structured. This included separating controllers from services, keeping data access in repositories, securing both HTTP and WebSocket interactions, organizing frontend pages with Thymeleaf and page-specific assets, and maintaining a clear repository structure.

3. Technology Stack and Why It Was Chosen

The project uses a stack that supports full lifecycle development rather than isolated coding exercises.

Technology stack used:

- `Java 17`: modern, stable language version for backend development
- `Spring Boot 4.0.3`: central framework for configuration, web, security, and application setup
- `PostgreSQL`: realistic relational database for production-style persistence
- `Spring MVC + Thymeleaf`: keeps frontend pages and backend in one maintainable application
- `Spring Security + JWT + BCrypt + Passay`: supports secure auth, token handling, password protection, and validation

- `Spring WebSocket + STOMP + SockJS`: enables live exchange chat
- `Maven`: build and dependency management
- `Docker + Docker Compose`: reproducible local environment and easier startup
- `GitHub Actions`: CI verification on push and pull request
- `Render`: cloud deployment support

This stack was chosen because it supports development, security, testing, collaboration, and deployment in one complete workflow.

4. Architecture and Project Structure

One of the biggest strengths of the repository is its clean architectural organization. The project follows a layered structure in which controllers manage requests, services contain business rules, repositories handle persistence, and DTOs keep API contracts separate from JPA entities.

The controller layer handles page routing, REST endpoints, and WebSocket entry points. The service layer contains the main application logic, including authentication flows, book handling, exchange validation, chat access, delivery processing, notification generation, and user-role operations. The repository layer uses Spring Data JPA interfaces to interact with the database. The entity layer represents the domain model.

This project structure directly supports maintainability. Thin controllers are easier to understand. Service-layer rules are easier to test and reuse. Repository concerns remain separated from business logic. DTOs protect the system from exposing raw entity design to every client.

The repository also includes a main `README.md`, which helps make the codebase easier to understand and maintain.

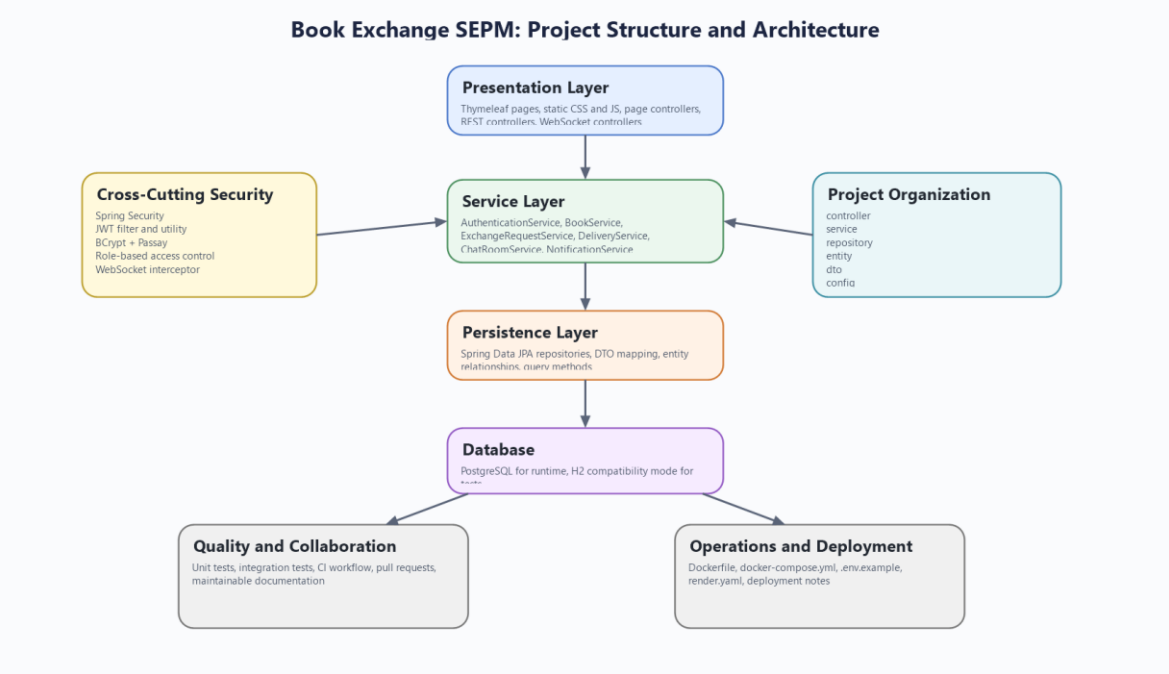

Figure 2. Layered architecture and package structure used in the project.

5. Security, Authentication, and Roles

Security was one of the main pillars of the project. The system uses Spring Security as the central place for controlling authentication and authorization instead of spreading security decisions across unrelated classes.

Authentication supports multiple use cases. Browser users can work through form login, while token-based API access is handled through JWT. Passwords are protected using BCrypt, and password quality is checked using Passay. The email verification flow adds another layer of realism because the system does not simply create an account and trust it immediately. That reflects a more production-aware user lifecycle.

Authorization is role-based. The project uses `ROLE_USER`, `ROLE_MODERATOR`, `ROLE_ADMIN`, and `ROLE_DELIVERY_MAN`. These roles shape access to endpoints and operations in meaningful ways. A normal user can manage books and requests, moderators can review exchange activity and remove inappropriate listings, admins can manage users and system sections, and delivery-role users can work with delivery assignments. The project also

combines role checks with ownership validation, which is important because secure design must consider both who the user is and what resource they are interacting with.

Another strong point is that security is not limited to normal REST requests. Exchange chat is protected through both service-layer access checks and the WebSocket channel interceptor.

6. REST API Design and Business Logic

The REST API is organized by domain rather than by generic technical grouping. Authentication, books, exchanges, users, wishlist subscriptions, notifications, and admin-related operations are separated into clear controller areas.

The more important part, however, is how logic is placed behind those endpoints. Business rules live in services such as `AuthenticationService`, `BookService`, `ExchangeRequestService`, `DeliveryService`, `ChatRoomService`, `UserService`, `WishlistService`, and `NotificationService`. This helps the project avoid one of the most common problems in student work: controllers becoming overloaded with all application logic.

Several examples show why this matters. Exchange creation validates ownership, availability, and duplicate-request rules. Approval and completion are not simple status updates; they are part of a controlled workflow. Delivery completion performs final ownership transfer. Wishlist notifications are triggered through an event-based path when book availability changes.

This approach gives the system stronger internal consistency and makes the project easier to test.

7. Use of Design Patterns

The project intentionally includes software design patterns where they improve clarity and maintainability.

The Strategy pattern is used in the book search module. Different search behaviors such as keyword, title, author, and genre matching are implemented through dedicated strategy classes and selected through a resolver. This keeps search logic modular and makes it easier to add new search modes in the future.

The Factory pattern appears in controlled object creation, especially through `UserFactory` and `ExchangeRequestFactory`. This reduces repeated initialization logic and keeps creation rules centralized. It also improves readability because object creation becomes intention-revealing instead of being repeated in multiple places.

The Singleton pattern is used in the book event manager that supports the notification flow around book availability. The goal is to keep one central event hub for publishing and subscribing to those events. In the context of this project, that is a reasonable design because it creates clear event coordination without adding infrastructure complexity that would be unnecessary for the current scale.

These patterns are meaningful because they are connected to actual project behavior and improve extensibility and clarity.

8. Full-Stack Integration with Thymeleaf and WebSocket

Book Exchange SEPM is not only a backend API project. It is a full-stack monolith with integrated UI, server-side rendering, REST communication, and real-time chat. Thymeleaf is used to render pages such as the landing page, browse page, book page, profile page, exchange page, delivery page, wishlist page, and admin sections. Each page is supported by focused CSS and JavaScript files, which keeps the frontend organized and easier to maintain.

This choice fits the project well. For a team of two, keeping frontend and backend inside one Spring Boot application reduces overhead and makes local development more straightforward.

WebSocket support adds another important engineering dimension. The project uses STOMP and SockJS to support exchange-specific live chat. The chat feature is tied to exchange participation, message history can be fetched through REST, and the security model remains active in the real-time channel.

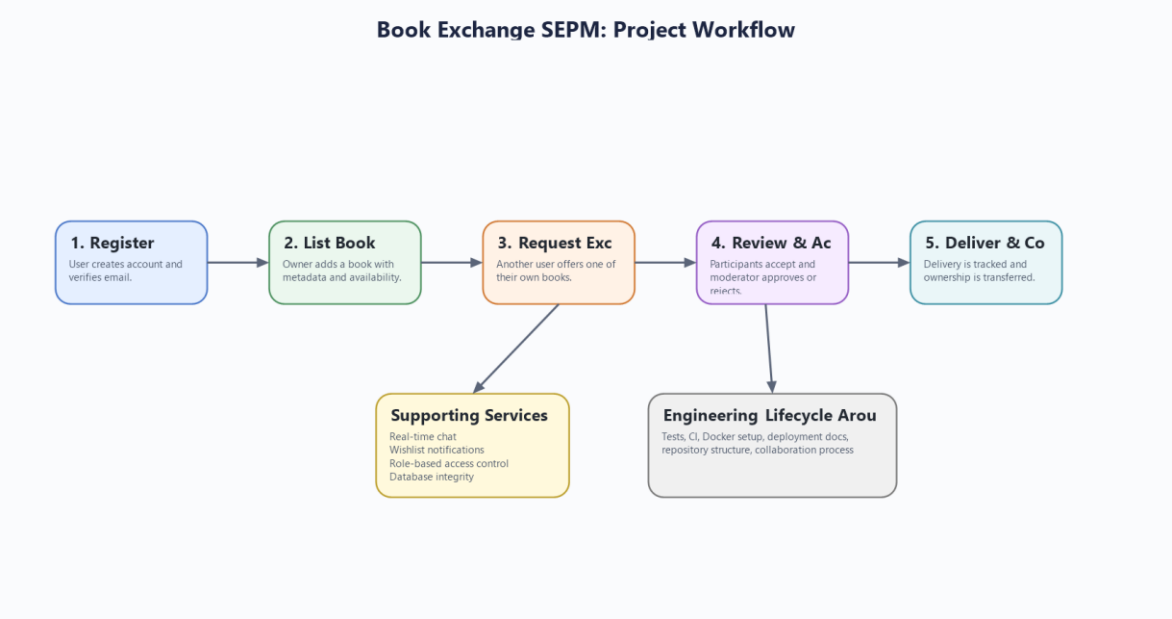

Figure 3. The system combines workflow handling with security, chat, notification, testing, and deployment support.

Together, Thymeleaf and WebSocket make the project much more representative of a real full-stack application.

9. Database Design and Workflow Thinking

The application's data model supports the project's workflow-heavy design. The most important entities include `User`, `Role`, `Book`, `ExchangeRequest`, `Delivery`, `ChatRoom`, `ChatMessage`, `WishlistSubscription`, and `UserNotification`.

This matters because the project is not just storing isolated records. The database supports a workflow in which books belong to users, exchanges connect multiple users and books, deliveries control completion, and chat is linked to a specific exchange context.

The project also includes compatibility and integrity initializers in the configuration layer. These classes help maintain schema consistency and repair evolving data assumptions. That is a thoughtful addition because it reflects the reality that software systems change over time and need help staying stable as the model evolves.

10. Testing and Quality Assurance

Testing is another major area where the project shows maturity. The repository contains unit tests, integration tests, security authorization tests, and smoke-level application validation.

The Maven build is configured so that Surefire runs unit tests, while Failsafe handles integration tests and application-level verification. The test suite includes service-level checks and end-to-end style verification around security and workflow behavior.

The use of H2 in PostgreSQL compatibility mode supports fast automated testing while staying reasonably close to the production-style relational model. Error handling is also standardized through the global exception handler, which improves both client behavior and testability.

11. GitHub Workflow, Team Collaboration, and CI

Because the project was completed by a two-person team, collaboration process mattered a great deal. That is why the repository includes a GitHub Actions CI workflow that runs on both `push` and `pull_request`.

The CI workflow is separated into unit testing and integration testing jobs. This provides a good base for pull-request-based development and aligns well with branch protection thinking. In practice, this means the team can use branches, submit pull requests, and rely on automated checks before merging.

The repository also helps teamwork through its structure. Environment templates, deployment configuration, and organized packages reduce confusion and make it easier for one collaborator to understand or continue the work of the other. For a project built by two people, that is a meaningful achievement.

12. Deployment, Docker, and Render Support

The project was prepared not just for coding and local presentation, but also for reproducible setup and deployment. Docker and Docker Compose make it easier to run the application together with PostgreSQL in a consistent environment. The `Dockerfile`, `docker-compose.yml`, and `.env.example` files support this process clearly.

Render deployment support is also included through `render.yaml`. This setup shows how the same project can move from local development to a hosted platform by using environment variables for database and runtime configuration. Even though the repository does not contain a full production CD pipeline, it is deployment-ready in a practical sense because the main packaging and environment decisions are already defined.

This is important because it shows the project was treated as something that should be runnable and maintainable beyond the IDE.

13. Conclusion

Book Exchange SEPM is best understood as an engineering-focused full-stack project rather than only as a feature-focused web application. Its importance comes from the way it brings together architecture, security, backend structure, full-stack integration, design patterns, testing, collaboration workflow, and deployment preparation within one coherent repository.

The project succeeds because it shows how a realistic domain can be used to practice real software engineering habits. Authentication and roles are handled carefully. Business logic is structured in services. REST APIs are organized clearly. WebSocket and Thymeleaf are

integrated into the same maintainable system. Testing and CI are part of the workflow. Docker and Render support deployment readiness. Documentation supports maintainability and teamwork.

For these reasons, the project demonstrates more than implementation effort. It demonstrates engineering discipline.